

Proposta Técnica para Pipeline Kanban Responsivo

Datacrazy

GABRIEL F. HERMENEGILDO

SUMÁRIO

1. Introdução.....	3
2. O que foi observado.....	3
3. Decisão de layout e navegação entre colunas.....	4
3.1. Por que não replicar o drag and drop horizontal do web.....	4
3.2. Modelo de dados e validação dos endpoints.....	5
3.3. Estados vazio e de erro.....	5
4. Gestos touch e movimentação de etapa.....	6
4.1. Reconhecimento de gestos.....	6
5. Reordenação de cards dentro da coluna.....	7
5.1. Por que não usar react-native-draggable-flatlist.....	7
5.2. Recomendação: react-native-reanimated-dnd.....	7
5.3. Alternativa: react-native-draglist.....	8
6. Atualização de etapa.....	8
7. Componentização do card.....	9
8. Performance.....	9
9. O que fica de fora da primeira versão.....	10
10. Resumo das decisões.....	11
11. Conclusão.....	12

1. Introdução

Este documento apresenta a proposta técnica para levar o pipeline em formato kanban, hoje disponível no CRM web, para o aplicativo mobile. A ideia não é reproduzir a interface do desktop em uma tela menor, mas adaptar a mesma operação para um contexto em que o usuário tem menos espaço de tela e interage principalmente por toque.

A solução parte da recomendação de manter React Native como base do aplicativo. Neste caso, isso permite construir a experiência para Android e iOS sem criar duas implementações independentes da mesma tela. A proposta também considera que o aplicativo está em React Native 0.82 com a Nova Arquitetura, premissa importante para as bibliotecas de navegação e gestos avaliadas.

O foco aqui é a implementação. Por isso, a proposta detalha a composição da tela, a navegação entre etapas, os gestos, a movimentação dos negócios, a reordenação dos cards, o tratamento de estado e erro, as bibliotecas consideradas, os cuidados de performance e o que eu deixaria fora da primeira versão.

2. O que foi observado

O kanban do CRM web organiza os negócios em colunas por etapa dentro de uma pipeline. O usuário pode ter mais de uma pipeline simultaneamente, e cada uma possui seu próprio conjunto de colunas. Essas colunas são configuráveis e podem ser adicionadas ou editadas pelo próprio usuário, então a implementação mobile não pode assumir um número fixo de etapas nem nomes fixos.

Cada card do web apresenta lead, produto, atendente, valor, data e tags. A movimentação entre etapas acontece por arrastar o card de uma coluna para outra. No aplicativo, algumas partes dessa operação já existem isoladamente: é possível criar um negócio a partir do lead escolhendo pipeline e etapa, e a tela do negócio já permite mover o negócio, editar o valor e marcar como ganho ou perdido.

O que falta, portanto, não é a capacidade de alterar a etapa de um negócio. O que falta é a visão de pipeline, que permite enxergar o funil e agir sobre vários negócios sem precisar abrir cada detalhe individualmente.

3. Decisão de layout e navegação entre colunas

No celular, não considero adequado manter todas as colunas visíveis lado a lado. A largura disponível não permite que o usuário leia os cards com conforto e ainda tenha espaço suficiente para interagir com eles. Minha proposta é exibir uma coluna por vez, ocupando a largura da tela, e permitir a navegação horizontal entre as etapas.

Para essa navegação, a escolha é o `react-native-pager-view`. A biblioteca utiliza `ViewPager` no Android e `UIPageViewController` no iOS por baixo, então o `swipe` horizontal se apoia em componentes nativos da própria plataforma em vez de ser simulado por um `ScrollView`. A versão considerada é a 9.x, e a partir da 7.x a biblioteca passou a exigir a Nova Arquitetura, o que está alinhado com a base proposta para o aplicativo.

No topo da tela, eu colocaria um indicador sincronizado com a página atual, podendo ser representado por abas ou pontos. O objetivo é deixar claro em qual etapa o usuário está e quantas etapas existem na pipeline. Isso também evita que o `swipe` seja a única forma de entender a posição atual dentro do funil.

3.1. Por que não replicar o `drag and drop` horizontal do web

No web, todas as colunas ficam visíveis ao mesmo tempo. Por isso, arrastar um card de uma coluna para outra acontece dentro do campo de visão do usuário. No mobile, a coluna de destino pode estar fora da área visível. Para manter o mesmo gesto seria necessário combinar o arraste do card com `autoscroll` horizontal enquanto o dedo continua pressionado.

Tecnicamente, isso é possível. Aplicativos de `kanban` consolidados utilizam esse padrão. O problema é o conjunto de casos de borda que passa a existir: velocidade do `autoscroll`, cancelamento do gesto no meio do caminho, mudança de direção, definição da área de destino e feedback visual sobre onde o card será solto.

Por isso, eu não colocaria esse comportamento como requisito da primeira versão. A primeira implementação usa uma ação mais simples e testável para mover entre etapas, mas o `drag and drop` cross-coluna com `autoscroll` fica registrado como uma evolução possível. A decisão de adiar não é por limitação técnica, e sim por custo e risco.

3.2. Modelo de dados e validação dos endpoints

Como as colunas são configuráveis por pipeline, o componente mobile deve renderizar dinamicamente a estrutura recebida da API. A referência é a mesma estrutura usada pelo web: pipeline selecionada, lista ordenada de colunas e negócios pertencentes a cada coluna. A princípio, isso não exige alterar o contrato da API, apenas adaptar a forma como os dados são apresentados.

Antes de implementar a tela, eu validaria os endpoints consumidos pelo mobile contra os endpoints do web. A análise encontrou casos de divergência para o mesmo dado e período, como uma seção de produtos que aparecia vazia no aplicativo e preenchida no web. Para o kanban, isso significa confirmar principalmente o conjunto de colunas e negócios, paginação e filtros de período.

Essa validação é importante porque uma inconsistência de API pode fazer uma interface correta parecer quebrada. Se a tela recebe apenas parte dos negócios ou uma lista de etapas diferente da usada no web, o problema não estará no componente de kanban.

3.3. Estados vazio e de erro

Como a navegação acontece uma coluna por vez, chegar a uma etapa sem negócios é um comportamento normal. Essa situação precisa ter um estado vazio próprio, com uma mensagem curta, como "Nenhum negócio nesta etapa". O objetivo é diferenciar claramente uma etapa sem registros de uma tela que não conseguiu carregar.

O estado de erro deve ser separado do estado vazio e apresentar uma mensagem legível, além de uma ação para tentar novamente. A análise do aplicativo encontrou fluxos em que erros aparecem como código bruto da API ou simplesmente não são apresentados. O kanban não deve repetir esse comportamento, principalmente porque uma falha silenciosa pode ser confundida com uma coluna realmente vazia.

4. Gestos touch e movimentação de etapa

Para mover um negócio de uma etapa para outra, eu usaria duas formas de acesso. A primeira é um botão de menu visível no próprio card, com ações como "Ver negócio" e "Mover para". A segunda é um toque longo no card, funcionando como atalho para a mesma ação.

Essa combinação resolve um problema de descoberta que já apareceu na análise do aplicativo. Foram observadas ações úteis escondidas atrás de interações sem indicação visual, como copiar telefone e email apenas ao tocar e acessar a agenda de atividades por um ícone sem sinalização suficiente. No kanban, não faria sentido depender de um gesto escondido para uma operação importante.

Ao escolher "Mover para", um bottom sheet reutilizando o padrão já presente no aplicativo apresenta as etapas disponíveis naquela pipeline. O usuário escolhe o destino e a movimentação acontece sem precisar manter o dedo pressionado enquanto a tela muda de coluna.

Essa solução também deixa uma evolução futura bem definida. Depois que a navegação por pager, os estados de erro e a movimentação básica estiverem estáveis, o drag and drop cross-coluna com autoscroll pode ser testado como uma segunda etapa.

4.1. Reconhecimento de gestos

Para o toque longo e outras interações de gesto, a base proposta é `react-native-gesture-handler`, considerada na versão 3.x, combinada com `react-native-reanimated` 4.x. As duas bibliotecas são mantidas pela Software Mansion e estão alinhadas com a Nova Arquitetura.

O Reanimated permite que a resposta visual do gesto, animações e feedback de interação aconteçam mais próximo da camada de UI, reduzindo a dependência da thread JavaScript para respostas que precisam ser imediatas. O `react-native-gesture-handler` fica responsável pelo reconhecimento e gerenciamento dos gestos.

Existe uma dependência importante de versão. O Reanimated 4 exige o pacote `react-native-worklets` como dependência separada, não deve coexistir com uma instalação do Reanimated 3 e suporta apenas a Nova Arquitetura e as três últimas versões do React

Native. Considerando React Native 0.82, o cenário é compatível, mas isso torna a manutenção da versão do projeto uma preocupação explícita.

O principal ponto de atenção da navegação é o conflito entre o swipe horizontal do pager e o scroll vertical da lista de cards. A regra proposta é usar o eixo dominante do movimento para decidir qual gesto responde: movimento predominantemente horizontal troca de etapa; movimento predominantemente vertical rola os cards. Esse comportamento precisa ser validado em aparelhos reais, tanto no Android quanto no iOS.

5. Reordenação de cards dentro da coluna

Reordenar cards dentro da mesma coluna é um caso diferente de mover um negócio entre etapas. Aqui, o gesto acontece dentro de uma área visível da tela, então o arrastar e soltar pode fazer parte da primeira versão sem exigir autoscroll horizontal entre páginas.

A escolha da biblioteca merece atenção porque o projeto está em React Native 0.82, com Nova Arquitetura e Reanimated 4. Uma biblioteca que funcione bem em versões antigas do ecossistema não é necessariamente uma boa escolha para essa base.

5.1. Por que não usar react-native-draggable-flatlist

react-native-draggable-flatlist é uma das bibliotecas mais conhecidas para listas arrastáveis e tem um histórico relevante de uso. Mesmo assim, eu não a escolheria para este projeto. A versão 4.0.3 está há cerca de dois anos sem publicação, o repositório possui issues abertas sem resposta desde 2024 e existe histórico documentado de incompatibilidades a cada versão maior do Reanimated.

Esse ponto pesa mais agora porque o Reanimated está na versão 4.x e utiliza uma arquitetura de worklets diferente das versões para as quais a biblioteca foi originalmente construída. Adotá-la em um aplicativo na versão 0.82 com Nova Arquitetura seria introduzir uma dependência justamente na região onde o ecossistema mudou mais.

5.2. Recomendação: react-native-reanimated-dnd

Minha escolha principal para a reordenação é react-native-reanimated-dnd. A biblioteca é construída sobre react-native-gesture-handler e react-native-reanimated, possui versão

2.0.0 considerada recente no levantamento e já foi adaptada para Reanimated 4 com react-native-worklets e Nova Arquitetura.

Ela oferece componentes para listas ordenáveis, que atendem diretamente ao caso de reordenação dentro da coluna. Outro ponto interessante é que também possui suporte a drag and drop entre áreas. Isso cria um caminho natural para uma evolução futura caso o drag and drop cross-coluna com autoscroll entre no roadmap.

A ressalva é que ela é mais nova e não possui o mesmo histórico de uso em produção que o react-native-draggable-flatlist. Ainda assim, considero que a compatibilidade com a stack atual pesa mais neste caso. Antes de fechar a dependência, eu faria uma prova de conceito com a versão exata do projeto.

5.3. Alternativa: react-native-draglist

Se o time preferir reduzir a dependência de Reanimated na implementação da lista, react-native-draglist é uma alternativa. A proposta da biblioteca é implementar listas arrastáveis utilizando React e React Native, com uma abordagem mais simples e sem colocar o Reanimated no caminho principal da reordenação.

A troca é consciente: existe menos espaço para incompatibilidades ligadas ao Reanimated, mas também há menos refinamento visual e menos recursos de animação. Como a reordenação não é a espinha da tela, esse risco fica relativamente isolado e a implementação pode ser substituída sem afetar a navegação por pager ou a movimentação entre etapas.

6. Atualização de etapa

Ao escolher uma nova etapa no menu, a movimentação deve ser otimista. O card muda de coluna imediatamente na interface, enquanto a requisição de atualização é enviada ao servidor em paralelo. Isso evita que o usuário precise esperar uma resposta de rede para enxergar o resultado da ação.

Se a API confirmar a alteração, o estado permanece como está. Se a requisição falhar, o card volta para a coluna original por meio de rollback e o usuário recebe um aviso claro sobre o que aconteceu, com possibilidade de tentar novamente.

Esse aviso é importante neste aplicativo porque a análise encontrou ações que falham silenciosamente. No kanban, esse problema seria ainda mais confuso: o usuário vê o card mudar de posição e depois voltar, sem saber se houve erro, perda de conexão ou outro problema. A mensagem deve explicar a falha em linguagem de usuário e não expor o código bruto retornado pela API.

A operação também deve usar um identificador único por tentativa de movimentação. Esse identificador permite tratar a operação de forma idempotente, evitando que uma retentativa causada por conexão instável produza uma segunda movimentação em vez de apenas confirmar a primeira. O mesmo princípio pode ser aplicado aos fluxos de envio já previstos para o multiatendimento.

7. Componentização do card

O card mobile deve mostrar apenas o que ajuda o usuário a decidir rapidamente qual negócio precisa de atenção. Minha proposta é manter nome do lead, valor e atendente, além do botão de menu. Produto e tags, que aparecem no card do web, podem continuar na tela de detalhe do negócio.

Essa redução não significa retirar informação do produto. Significa colocar cada informação no contexto em que ela é mais útil. No celular, o objetivo do card é permitir uma varredura rápida. Colocar informação demais reduz a quantidade de cards visíveis e torna mais lenta a identificação do próximo negócio que precisa de ação.

Cada card pode ser implementado como um componente isolado e memoizado com `React.memo`, combinado com props estáveis e virtualização da lista. Isso reduz renderizações desnecessárias quando apenas um card muda de estado. É importante tratar isso como uma otimização e não como garantia absoluta de que a lista inteira nunca será renderizada novamente.

8. Performance

A listagem de negócios dentro de cada coluna começa com `FlatList` e virtualização, renderizando principalmente os cards visíveis. Para a primeira versão, eu não adicionaria uma biblioteca de lista mais complexa apenas por expectativa de performance.

FlashList, mantida pela Shopify, pode ser avaliada posteriormente se os testes com volume real de negócios mostrarem que a FlatList não atende bem. O critério deve ser medição em aparelho de entrada, com quantidade de dados próxima do uso real, e não uma percepção genérica de que outra biblioteca é mais rápida.

Essa decisão também acompanha o diagnóstico geral do aplicativo. Não foram encontrados indícios de que lentidão de renderização seja hoje o principal problema. Os pontos mais relevantes estão relacionados a dados, erros e escopo de funcionalidade. Nesse contexto, otimizar antecipadamente a lista antes de medir seria investir esforço em um gargalo ainda não comprovado.

9. O que fica de fora da primeira versão

Registrar o que não entra é tão importante quanto registrar o que entra. Isso evita que uma funcionalidade ausente seja interpretada como esquecimento e ajuda a manter o primeiro ciclo de desenvolvimento concentrado no que realmente precisa estar estável.

- Drag and drop cross-coluna com autoscroll. Fica adiado. Eu revisitaria depois que a navegação por pager estiver estável e testada e caso o uso real mostre que o menu de mover não atende bem. A biblioteca recomendada para reordenação já possui suporte a arraste entre áreas, então essa evolução não exigiria necessariamente trocar de dependência.
- Edição de card diretamente no kanban. O fluxo continua direcionando para a tela de detalhe do negócio, que já existe e concentra as operações de edição. Duplicar edição em dois lugares aumentaria a superfície de bugs sem um ganho claro.
- Criação de pipeline e edição de colunas. Essas operações são de configuração e podem continuar na web neste primeiro momento.
- Filtros avançados sobre o kanban. A primeira versão respeita a pipeline selecionada e apresenta os negócios das etapas. Filtros mais exploratórios podem continuar seguindo a lógica de atalhos rápidos no mobile e filtragem completa na web.

10. Resumo das decisões

Decisão	Escolha	Motivo principal
Layout de colunas	Uma coluna por vez, largura total, pager horizontal	A tela do celular não comporta várias colunas com leitura confortável.
Navegação entre etapas	react-native-pager-view 9.x	Usa ViewPager e UIPageViewController nativos e está alinhado à Nova Arquitetura.
Reconhecimento de gesto	react-native-gesture-handler 3.x + react-native-reanimated 4.x	Bibliotecas alinhadas à Nova Arquitetura e adequadas para gestos e resposta visual.
Conflito de gesto	Eixo dominante decide	Movimento horizontal troca etapa; movimento vertical rola os cards.
Mover entre colunas	Menu no card + toque longo + bottom sheet	Mais simples de testar e mais descobrível que drag and drop cross-coluna.
Reordenar na coluna	react-native-reanimated-dnd , com react-native-draglist como alternativa	Compatibilidade com a stack atual, com caminho de evolução para drag and drop entre áreas.
Atualização de etapa	Otimista com rollback e idempotência	Resposta rápida na interface e segurança em retentativas.
Conteúdo do card	Lead, valor e atendente	Prioriza varredura rápida e mantém detalhes na tela própria.

Decisão	Escolha	Motivo principal
Lista de cards	FlatList com virtualização	Começa simples; FlashList só entra se medição indicar necessidade.
Estados vazio e erro	Visuais distintos e explícitos	Evita confundir ausência de negócios com falha de carregamento.

11. Conclusão

A proposta é adaptar o kanban ao uso real do celular, em vez de tentar reproduzir a disposição do web em uma tela menor. Uma coluna por vez, navegação por swipe e cards mais enxutos deixam a operação adequada à largura disponível, enquanto o menu de movimentação garante que a principal ação continue fácil de descobrir.

Para a implementação, a base fica concentrada em React Native 0.82 com Nova Arquitetura, react-native-pager-view para navegação, react-native-gesture-handler e react-native-reanimated para gestos, react-native-reanimated-dnd para reordenação e FlatList para a listagem inicial. A atualização otimista com rollback e idempotência mantém a interação rápida sem abrir mão da consistência com o servidor.

O ponto central da proposta é não colocar toda a complexidade do kanban web na primeira versão. O drag and drop cross-coluna com autoscroll é tecnicamente possível e pode ser incorporado depois, mas a primeira entrega deve priorizar uma interação mais simples, testável e previsível. Com essa base estável, fica mais fácil medir o uso real e decidir quais refinamentos realmente valem o custo.